

Day 4 - Functions: Declarations, Arrow Functions, Scope, Closures, this

Day 4 — Functions: Declarations, Arrow Functions, Scope, Closures, `this`

Objectives

- Write functions three different ways: declarations, expressions, and arrow functions — and know when each is preferred
- Understand default parameters and rest parameters
- Understand scope (`let/const` block scope vs the older, looser `var` function scope)
- Understand closures
- Understand `this` — genuinely one of the trickiest ideas in JavaScript, and get a solid, correct mental model for it today

Three ways to write a function

Function declarations

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(2, 3)); // 5
```

This should look familiar from every example so far. A useful, JavaScript-specific quirk: function declarations are "hoisted" — meaning you can actually call one *before* the line where it's written in your file, because JavaScript processes all function declarations first, before running the rest of the code top to bottom:

```
console.log(greet()); // works fine, even though greet is "defined" below this line  
  
function greet() {  
  return "hi";  
}
```

Function expressions

```
const add = function (a, b) {  
  return a + b;  
};
```

Here, a function is created and immediately stored in a variable, `add` — this is called a **function expression** (as opposed to a declaration) because the function itself is just a value being assigned, exactly like assigning a number or a string. Unlike declarations, function expressions are **not** hoisted — you cannot call `add()` before this line runs.

Arrow functions — the modern, most common style for short functions

```
const add = (a, b) => {
  return a + b;
};

const addShort = (a, b) => a + b;    // "implicit return" -- no braces, no `return` keyword, just one expression

const square = (n) => n * n;        // one parameter -- parentheses are optional here, but keep them for clarity
const sayHi = () => console.log("hi"); // zero parameters -- parentheses ARE required
```

Arrow functions are the modern, preferred way to write short functions, especially ones passed as arguments to other functions (recall yesterday's `.map((n) => n * 2)`). When the function body is just one single expression, you can drop the curly braces and the `return` keyword entirely — the value of that one expression is automatically returned; this is called an **implicit return**, and it's an extremely common style you'll see everywhere in real JavaScript and TypeScript code.

When to use which: reach for arrow functions by default, especially for short, one-off functions (passed to `.map/.filter/.reduce`, event handlers, and similar). Use a plain `function` declaration for a "main," named function that represents a core, standalone piece of logic in your file — it reads slightly clearer at the top level, and gets the hoisting benefit above. There's one more, genuinely important difference between arrow functions and regular functions — how they handle `this` — covered later in this lesson, and it's actually the main technical reason arrow functions are often preferred, not just a style preference.

Default and rest parameters

```
function greet(name, greeting = "Hello") { // default value, used if the caller doesn't supply one
  console.log(`${greeting}, ${name}!`);
}

greet("Ana"); // "Hello, Ana!"
greet("Bo", "Good morning"); // "Good morning, Bo!"

function sum(...numbers) { // REST parameter -- gathers any number of arguments into one array
  return numbers.reduce((total, n) => total + n, 0);
}

console.log(sum(1, 2, 3, 4)); // 10 -- called with any number of arguments at all
```

A default parameter's value is computed fresh, at the moment the function is called, each time it's actually needed — unlike Python's infamous mutable-default-argument trap (if you've done the Python track), JavaScript recomputes a default expression on every call, so this specific bug doesn't exist here in the same way.

Scope — which parts of your code can see which variables

Variables declared with `let` or `const` are **block-scoped** — they only exist inside the nearest enclosing pair of curly braces `{ }`:

```
if (true) {
  const message = "hello";
  console.log(message); // "hello" -- fine, we're still inside the block
}
```

```
console.log(message); // ReferenceError: message is not defined -- outside the block, it d
```

This is one of the concrete reasons `let/const` are preferred over the older `var`, which is **function-scoped** instead — it leaks out of `if/for/while` blocks and is only contained by an enclosing `*function*`, which historically caused a lot of confusing bugs (a variable you thought was safely "local" to one small block turning out to still be visible much further away).

```
function calculate() {
  const result = 42;
  return result;
}

calculate();
console.log(result); // ReferenceError: result is not defined -- result only existed inside calcul
```

Closures — a function that remembers where it was created

Exactly the same underlying idea as the Python track's closures, if you've seen that — a function defined *inside* another function can "remember" variables from the outer function, even after the outer one has already finished running:

```
function makeMultiplier(factor) {
  return function (n) {
    return n * factor; // `factor` is remembered from the ENCLOSING function
  };
}

const double = makeMultiplier(2);
const triple = makeMultiplier(3);

console.log(double(5)); // 10
console.log(triple(5)); // 15
```

Each call to `makeMultiplier` creates its own separate, private `factor`, which is why `double` and `triple` don't interfere with each other, despite being built from the same inner function shape. You'll rely on this exact mechanism constantly once you're writing more advanced JavaScript — it's how a huge amount of real-world JavaScript manages private, hidden state.

`this` — genuinely the trickiest thing in today's lesson, so read carefully

`this` is a special keyword that refers to "whatever object this function is currently being called as a method of" — but, crucially, **its value is decided by *how* a function is called, not by where the function was written.** This is different from most other languages' equivalent concept, and it's the single most common source of confusing JavaScript bugs for people learning the language.

```
const person = {
  name: "Ana",
  greet: function () {
    console.log(`Hi, I'm ${this.name}`);
  },
};
```

```
person.greet();    // "Hi, I'm Ana" -- `this` refers to `person`, because greet was called AS person
```

Here, `this` correctly refers to `person`, because the function was called using `person.greet()` — calling a function "through" an object like this is exactly what sets `this` to that object. But watch what happens if you take that same function out and call it differently:

```
const greetFunction = person.greet;
greetFunction();    // "Hi, I'm undefined" -- `this` is NOT person anymore!
```

`this` is no longer `person`, because this time the function was called plainly, on its own, with no object in front of the dot — so `this` doesn't refer to `person` at all anymore. **This exact scenario is extremely common in real bugs** — for instance, passing `person.greet` as a callback to something else (like an event handler, which you'll eventually meet) silently loses its connection to `person`.

Arrow functions handle `this` completely differently — and this is the real, practical reason they're preferred`

Arrow functions do **not** get their own `this` at all — instead, they simply use whatever `this` was already in the surrounding code where the arrow function was written (this is called "lexical `this`," meaning it's determined by *where the code is written*, not by *how it's later called* — the opposite of a regular function's `this`):

```
const person = {
  name: "Ana",
  hobbies: ["reading", "hiking"],
  printHobbies: function () {
    // regular function used here deliberately, so `this` correctly refers to `person` inside it
    this.hobbies.forEach((hobby) => {
      // arrow function here -- it has NO `this` of its own, so it uses `printHobbies`'s `this`
      console.log(`${this.name} enjoys ${hobby}`);
    });
  },
};

person.printHobbies();
// Ana enjoys reading
// Ana enjoys hiking
```

If that inner function had instead been written as a regular `function` (not an arrow function), `this` inside it would NOT be `person` — it would be `undefined` (in modern JavaScript's strict mode) or something else entirely unrelated, because `.forEach()` calls its given function plainly, without any object in front of it, exactly like the `greetFunction()` example above. This exact distinction — "does `this` refer to what I expect, or not" — is precisely why arrow functions became so popular for anything nested inside a method: they simply inherit the correct, expected `this` from their surroundings, with no surprises.

A practical rule of thumb for today: when defining a method directly on an object (or, starting tomorrow, inside a class), and it genuinely needs access to that object's own data via `this`, write it as a regular function (or, in classes, the normal method syntax you'll see tomorrow). When writing a smaller function *nested inside* that method — a callback passed to `.forEach/.map/similar` — prefer an arrow function, specifically so it correctly inherits the outer `this` rather than losing it.

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js`.